Bash it up!

Get started with writing your own Bash scripts

Jayesh Limaye

Bash—an acronym for Bourne-again Shell, is a Unix shell written for the GNU Project and is the default shell for Linux and Mac OS X. The beauty of Bash scripts is that they let you accomplish a group of tasks quickly and without much intervention from the user—much like batch files in Windows. The syntax is similar to structured programming languages like C, and will be easier to understand for people with prior knowledge of any such programming language. We assume that you are familiar with GNU/Linux commands and with the fundamental concepts of programming.

Saving And Running Your Scripts

You can use any text editor to write your Bash scripts—any Unix-based OS you work on should have Vim installed. Save the scripts with the .sh extension anywhere on your computer. After saving it, open the terminal (as root, if necessary) and go to the folder where you saved the file. To run your script, type **bash yourscriptname.sh** in the terminal prompt.

A Very Simple Script

The first thing that anyone new to programming learns is how to print "Hello World" on the screen:

```
echo Hello World
```

This is a one line script where the action performed by the script is to print "Hello World" on the computer screen.

Variables, Functions and If

Variables can be used in Bash as in any other programming language. It is simpler here, since there are no data types to worry about—a variable can be a number, character or string. They need not be declared as in structured languages; they're created as soon as you assign a value to them.

Variables are of two types: *global*, which are recognised throughout the program, and *local*, whose recognition is limited to a function. If you have a block of code that you may want to execute a number of times, you write a function instead of the entire block all over again, saving time and resources in the bargain. Take a look at the example below:

```
#!/bin/bash
HELLO=Hello
function hello
{
local HELLO=World
echo $HELLO
}
echo $HELLO
hello
echo $HELLO
```

In the second line, the global variable **HELLO** is assigned a string value **"Hello"**. The third line declares a function called **"hello"**, which runs the code between the braces every time the function is called. Here a local variable **HELLO** is assigned a string value **"World"** which is valid only during the execution of the function **"hello"**. Notice that we have used the common name **HELLO** for both Local and Global variables just to show that this is possible. In the sixth line, the function declaration is complete. Line 7 displays **"Hello"** on the screen; in the next line, the function **"hello"** is called which prints **"World"** on the screen. The ninth line, however, prints **"Hello"** again. The string **"World"** is stored in the local variable **HELLO** but does not overwrite the value of the global variable of the same name.

You can add comments to your script to make it easier others to understand—any line starting with a "#" is interpreted as a comment and is not processed.

The "if" command lets you decide the direction in which the script should proceed depending on conditions you set. The syntax is as below:

```
if condition
then
#condition is true
execute all commands up to else statement
#The else block is optional
else
 #condition is not true
 execute all commands up to fi
fi
```

Some Basic Tasks

1. Renaming multiple files

The following script renames all the files with extension .txt to .bak.

```
#Use the "for" structure to run through all files
for f in *.txt;
do
mv "$f" "${f%.txt}.bak";
done
```

Similarly, to remove the prefix "prefx" from a group of files, the following script is used.

```
for f in prefx-*;
do
mv "$f" "${f#prefx-}";
done
Here ${f#prefx-} removes the prefix "prefx" from the filename.
```

2. Backing up files modified within the past 24 hours

Here, we will back up the files created in the current directory within the past 24 hours in a ".tar.gz" file.

```
#!/bin/bash
FILEBACKUP=backup-$(date +%m-%d-%Y)
incrback=${1:-$ FILEBACKUP}
tar cvf– `find . -mtime -1 -type f -print` > $incrback.tar
gzip $incrback.tar
echo "The directory $PWD has been backed up in archive file \"$incrback.tar.gz\"."
```

Line 2 of this script embeds the date in the backup filename. Line 3 introduces the string variable **incrback** as the filename of the archive. Line 4 tar condenses the current directory into a single tar file, while the fifth line gzip compresses the resultant tar file to the final tar.gz output file. The last line prints to the console that the current directory has been backed up to the archive file.

3. Mount and unmount drives as you please

You may wish to mount and/or unmount your drives swiftly and at one go:

```
# mounting the drives
# This mounts a partition of your hard drive
mount /dev/hda2 /hd -t ext3
# This mounts the floppy
mount /dev/fd0 /mnt/floppy
# This mounts the CD-ROM
mount /dev/cdrom /mnt/cdrom
```

And for unmounting:

```
# unmounting the drives
umount /dev/hda2
umount /dev/fd0
umount /dev/cdrom
```

Ready to Bash?

We hope that you've warmed up to the concept of Bash scripting by now. This is barely the surface—Bash scripting becomes more interesting and powerful as you delve deeper, and practice is the best way to take your Bash programming skills to new levels.■

jayesh_limaye@thinkdigit.com